

Report

1.01

Customer

Exactly



Audit 10

27 March 2026

Contents

1. Document Revisions	3
2. Introduction	3
3. System Overview	4
4. Methodology	4
5. Findings	6
5.1. Minor.....	6
6. Conclusion	8
About Us	9
Contacts	9

1. Document Revisions

#	Date	Author	Description
---	------	--------	-------------

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

ABDK Consulting is pleased to present the technical audit report for Audit 10, conducted on behalf of Exactly. This report details our findings and analysis of the specified code, with the goal of enhancing its security, efficiency, and overall integrity.

This audit focused on the changes introduced in the final commit of the Exactly protocol (<https://github.com/exactly/protocol/commit/e02a973>). Our examination encompassed the implementation of `AccessControlUpgradeable` with a `BRIDGE_ROLE` for cross-chain minting and burning functionalities, the integration of EIP-7802 for Superchain interoperability, and the addition of mint/burn functions for Hyperlane's HypXERC20 bridge. Additionally, we reviewed the new `initialize2(address admin_)` function with `reinitializer(2)` for setting up `AccessControl` and granting the `DEFAULT_ADMIN_ROLE` to the timelock.

The audit was performed by Dmitry Khovratovich and Mikhail Vladimirov. Their expertise in smart contract security and blockchain technology has been instrumental in providing a thorough and insightful assessment.

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

The audited diff is <https://github.com/exactly/protocol/compare/main...e02a973>

/periphery/
EXA.sol.diff

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the Audit 10 project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment

The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis

Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis

For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis

The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical

Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major

Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate

Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor

Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the Audit 10 codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Minor	4	0

Fixed 0 out of 4 issues reported.

5.1. Minor

CVF-1: acknowledged

Category: **readability**

Here a business-level property `symbol` is used as a technical low-level flag to tell, if the contract was initialized. This is a bad practice.

Recommendation

Use a dedicated flag or the `_getInitializedVersion` function provided by the "Initializable" base contract.

Client Comment

`_getInitializedVersion()` can't be used here: the `reinitializer(2)` modifier sets `= 2` before the function body executes (<https://github.com/OpenZeppelin/openzeppelin-contracts-upgradeable/blob/v4.9.6/contracts/proxy/utils/Initializable.sol#L121>), so it returns 2 regardless of whether `initialize` (version 1) ever ran. A dedicated flag would add a storage slot for a one-time check. `symbol()` reads existing ERC20 state set by `initialize` at no extra cost.

/periphery/EXA.sol.diff

```
35 +   if (bytes(symbol()).length == 0) revert NotInitialized();
```

CVF-2: acknowledgedCategory: **behavior**

Delegating to `crosschainMint` here means that the `CrosschainMint` event will be emitted and the `BRIDGE_ROLE` check will be performed. Both seem odd for a normal, i.e. non cross-chain mint.

Recommendation

Either remove the non-crosschain `mint` function, or delegate directly to `_mint` after performing proper authorization checks. Alternatively, if there is a valid reason to have `mint` behavior identical to `crosschainMint`, clearly explain this reason in a documentation comment.

Client Comment

By design. `mint/burn` exist solely for Hyperlane's `HypXERC20` router, which expects this interface. There is no non-bridge `mint/burn`, the total supply is fixed across all chains. Delegating to `crosschainMint/crosschainBurn` ensures the `BRIDGE_ROLE` check and `CrosschainMint/CrosschainBurn` events are always enforced, regardless of which bridge calls.

/periphery/EXA.sol.diff

```
53 + function mint(address to, uint256 amount) external {
54 +   crosschainMint(to, amount);
```

CVF-3: acknowledgedCategory: **behavior**

Delegating to `crosschainBurn` here means that the `CrosschainBurn` event will be emitted and the `BRIDGE_ROLE` check will be performed. Both seem odd for a normal, i.e. non cross-chain burn.

Recommendation

Either remove the non-crosschain `burn` function, or delegate directly to `_burn` after performing proper authorization checks. Alternatively, if there is a valid reason to have `burn` behavior identical to `crosschainBurn`, clearly explain this reason in a documentation comment.

Client Comment

By design. `mint/burn` exist solely for Hyperlane's `HypXERC20` router, which expects this interface. There is no non-bridge `mint/burn`, the total supply is fixed across all chains. Delegating to `crosschainMint/crosschainBurn` ensures the `BRIDGE_ROLE` check and `CrosschainMint/CrosschainBurn` events are always enforced, regardless of which bridge calls.

/periphery/EXA.sol.diff

```
57 + function burn(address from, uint256 amount) external {
58 +   crosschainBurn(from, amount);
```

CVF-4: acknowledgedCategory: **readability**

Declaring top-level errors in a file named after a contract make is harder navigating through code.

Recommendation

Move the error declarations into the contract or move them into a separate file.

Client Comment

Project convention. File level errors are used consistently across the codebase.

/periphery/EXA.sol.diff

```
77 +error NotInitialized();  
78 +error NotProxyAdmin();  
79 +error ZeroAddress();
```

6. Conclusion



About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

Email

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

[@ABDKConsulting](https://twitter.com/ABDKConsulting)

LinkedIn

company/abdk-consulting